

VNUHCM – UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



Circuit-Based Feature-Modular Clean Architecture

FIELDFLOW

Thành viên:

1. Trần Đình Quốc Thắng (24125079)
2. Trần Đăng Lê Huy (24125057)
3. Bùi Trọng Tuấn Linh (24125066)
4. Võ Quốc Linh (24125065)

TP. Hồ Chí Minh, 07/2026

Contents

1	Introduction	3
1.1	Problem	3
1.2	Motivation	3
1.3	Objectives	3
1.4	Scope	4
2	Background	4
2.1	The Dependency Rule	4
2.2	Layers and their Android counterparts	5
2.3	Dependency inversion and ports/adapters	5
2.4	Slack Circuit as the presentation pattern	6
2.5	Gradle modules as compile-time enforcement	6
3	Technical Approach	7
3.1	Architecture overview	7
3.2	Boundary rules	8
3.3	The domain layer	8
3.3.1	Business rules, and why they live here	8
3.3.2	Ports	10
3.3.3	Orchestration	11
3.4	The data layer	12
3.5	The presentation layer	13
3.6	The composition root	14
3.7	Offline-first	15
4	Demo and Case Study	16
4.1	The application	16
4.2	End-to-end trace: completing an inspection	16
4.3	Verification method	17
4.4	Results	18
4.5	Case study: replacing an adapter	19
5	Evaluation	20
5.1	Advantages observed	20
5.2	Limitations and costs	20
5.3	Trade-offs	22
5.4	Alternatives considered	22
5.5	Conditions for use	23
6	Conclusion	23
6.1	Key findings	23
6.2	Lessons learned	24

7 AI Usage Statement	24
References	26
A Building and Running FieldFlow	26
B Key Files for the Demonstration	27

1 Introduction

1.1 Problem

Android applications tend to degrade in a characteristic way. Business rules are written where they are first needed, inside an `Activity`, a composable, a `ViewModel`, or even inside a SQL query, because that is the shortest path to a working screen. Over time, three symptoms appear:

1. **Rules become untestable.** A validation rule that lives inside a composable cannot be verified without an emulator or a Robolectric runtime, so it is usually verified by hand, or not at all.
2. **Rules become duplicated or lost.** When the same rule is needed on a second screen, it is copied. When storage changes, the copy in the DAO silently disappears.
3. **Change becomes expensive.** Replacing a persistence library, a UI toolkit, or an export format touches code across the whole project because there is no boundary that says where infrastructure stops and policy begins.

The underlying cause is not a lack of discipline but a lack of an *enforced* dependency direction. Naming conventions and package structure express intent, but the compiler does not check them, so any file can import any other file.

1.2 Motivation

Clean Architecture, as formulated by Robert C. Martin, proposes a single organising principle — the *Dependency Rule* — that source-code dependencies must point inward, toward higher-level policy. Applied to Android, this means that business rules must not know that Android, Jetpack Compose, or Room exist.

The motivation for this seminar is to move that principle from a diagram to something *verifiable*. We therefore did not build a toy example. We built and measured a real multi-module Android application, **FieldFlow**, in which the architecture is enforced by the Gradle module graph rather than by convention, and in which every claimed benefit is backed by a file, a build script, or a passing test.

1.3 Objectives

1. Explain the Dependency Rule, dependency inversion, and the ports-and-adapters pattern in the concrete vocabulary of an Android project.
2. Show how Gradle modules convert an architectural convention into a *compile-time* constraint.
3. Demonstrate a complete end-to-end flow (UI → presenter → use case → port → adapter → Room) in working code.
4. Show that business rules are testable on a plain JVM without any Android dependency.

5. Show that infrastructure adapters are replaceable without touching the domain or the feature modules.
6. Evaluate the approach honestly: measure its cost, state its limits, and say when a simpler architecture would be the better engineering choice.

1.4 Scope

In scope. The architecture of the FieldFlow codebase as it exists in the submitted source tree: 15 Gradle modules, a pure Kotlin/JVM domain layer, a Room-backed data layer, eight Slack Circuit feature modules, an application-level composition root, and 187 automated unit tests (plus 4 instrumented tests run on a Gradle-managed device in CI).

Out of scope. FieldFlow is a functional prototype and this report does not claim otherwise. Authentication, a real backend, production cloud synchronisation, push notifications, report scheduling and e-mail delivery, full multi-section template authoring, and active/archive lifecycle controls are not implemented. The pending-sync queue is drained at runtime, but by a deterministic fake adapter standing in for a server that does not exist. Instrumented UI coverage is four navigation and draft-persistence tests, so most screen-level behaviour is still confirmed by hand. These exclusions are product-scope decisions and do not affect the architectural claims, but they are stated so the evaluation in Section 5 can be read fairly.

2 Background

2.1 The Dependency Rule

Clean Architecture organises a system as concentric circles. The inner circles contain policy (what the software *does*); the outer circles contain mechanisms (how it talks to the world). The Dependency Rule states:

Source code dependencies must point only inward, toward higher-level policies. Nothing in an inner circle can know anything at all about something in an outer circle.

Concretely: an inner-circle file must not contain an `import` of an outer-circle package. The domain may not import Room; Room-facing code may import the domain.

2.2 Layers and their Android counterparts

Clean Architecture circle	FieldFlow module	Contents
Entities (enterprise rules)	<code>:domain</code>	<code>InspectionSession</code> , <code>InspectionTemplate</code> , <code>InspectionScore</code> , <code>MaintenanceIssue</code>
Use cases (application rules)	<code>:domain</code>	<code>ValidateInspectionUseCase</code> , <code>CompleteInspectionUseCase</code> , <code>CalculateInspectionScoreUseCase</code>
Interface adapters	<code>:feature:*</code> , <code>:data</code>	Circuit presenters and UI state; Room repository adapters and mappers
Frameworks & drivers	<code>:core:database</code> , <code>:app</code>	Room database, DAOs, migrations, Android <code>Context</code> , file storage

Table 1: Mapping of Clean Architecture circles onto FieldFlow’s Gradle modules.

Entities and use cases share one module here. That is a deliberate simplification: FieldFlow is a single application, so there is no second application that would need to reuse the entities independently. The *boundary that matters*, between policy and mechanism, is still a hard module boundary.

2.3 Dependency inversion and ports/adapters

The Dependency Rule appears to create a paradox: a use case must *save* data, but saving is an outer-circle concern. Dependency inversion resolves it. The inner circle declares an interface expressing what it needs (a *port*); the outer circle provides the implementation (an *adapter*). The source-code dependency now points inward even though control flow points outward.

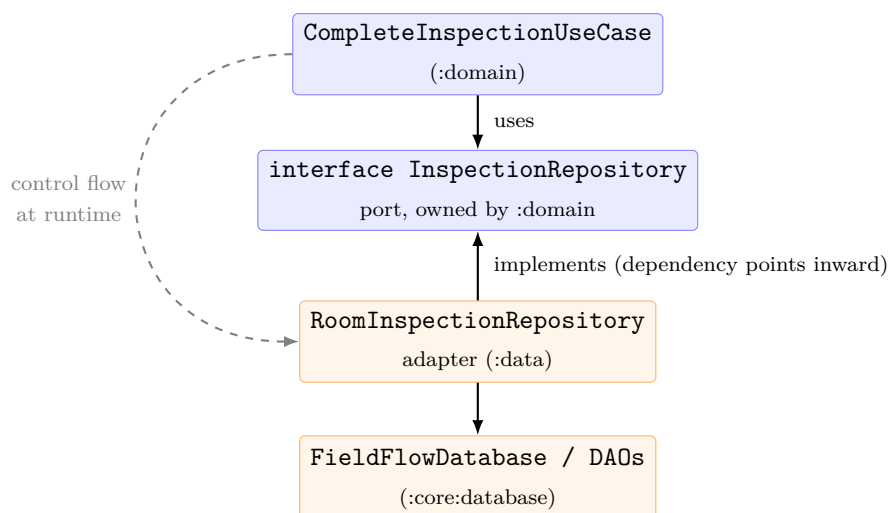


Figure 1: Dependency inversion. The arrow from `:data` to `:domain` is a compile-time dependency; the dashed arrow is runtime control flow. They point in opposite directions, which is exactly the point.

2.4 Slack Circuit as the presentation pattern

FieldFlow structures its presentation layer with **Slack Circuit** 0.33.1, a Compose-native unidirectional data-flow framework. Circuit defines four collaborating pieces:

`Screen` → `Presenter` → `immutable CircuitUiState` → `CircuitUiEvent` → `Compose Ui`

A `Screen` is a `Parcelable` navigation contract; a `Presenter` is a `@Composable` function that observes flows and emits immutable state; the `Ui` renders state and pushes events back through an `eventSink`.

It is important to be precise about what Circuit is *not*. Circuit is not a replacement for Clean Architecture, and it is not "just MVVM with new names". Circuit organises the *inside* of the outermost presentation circle. Clean Architecture still decides which circle owns business policy. The two are orthogonal, and this report treats them as such.

2.5 Gradle modules as compile-time enforcement

This is the mechanism that distinguishes an enforced architecture from a documented one. In a single-module project, "the UI must not touch the database" is a rule a human must remember. In a multi-module project, the same rule is expressed as an *absence* in a build file:

```
dependencies {
    implementation(platform(libs.androidx.compose.bom))
    implementation(project(":core:designsystem"))
    implementation(project(":core:navigation"))
    implementation(project(":domain"))
    // no ":data", no ":core:database" -- Room types cannot be imported here
    // ... (UI libraries omitted: Compose, Circuit, Coroutines) ...

    testImplementation(project(":core:testing"))
    // ... (Test libraries omitted) ...
}
```

Listing 1: `feature/inspection/build.gradle.kts` — the inspection feature can see the domain and shared UI, but `:data` and `:core:database` are simply not on its compile classpath.

An attempt to write `import com.topic11.cs426.data.RoomInspectionRepository` inside `:feature:inspection` is a compilation error, not a code-review comment. The architecture is checked by the build on every commit.

Domain purity is enforced by the same mechanism, one step more strictly:

```
plugins {
    'java-library'
    alias(libs.plugins.kotlin.jvm)
}

dependencies {
    api(libs.kotlinx.coroutines.core) // the only production dependency

    testImplementation(project(":core:testing"))
    testImplementation(libs.junit)
    testImplementation(libs.kotlinx.coroutines.test)
}
```

Listing 2: `domain/build.gradle.kts` — `:domain` is a `java-library`, not an Android library. The Android SDK, Compose, Circuit and Room are not available to it at all.

Note that `:domain` has exactly one production dependency: `kotlinx-coroutines-core`. It is permitted because the repository ports are reactive and return `Flow`; it introduces no framework coupling and no Android coupling.

3 Technical Approach

3.1 Architecture overview

FieldFlow implements what we call **Circuit-Based Feature-Modular Clean Architecture**: Clean Architecture supplies the dependency direction, feature modularisation supplies ownership and compile-time isolation, Circuit structures the presentation layer, and offline-first Room adapters supply persistence.

The application domain is facility and asset inspection. An inspector picks an asset, selects an inspection template, answers checklist items (pass / fail / not-applicable / measured value, with notes and evidence), saves drafts, reviews, and completes the inspection. Completion triggers validation, weighted scoring, automatic creation of maintenance issues for critical failures, and scheduling of the next inspection. This is a genuine domain which is what makes it a fair subject for a Clean Architecture case study.

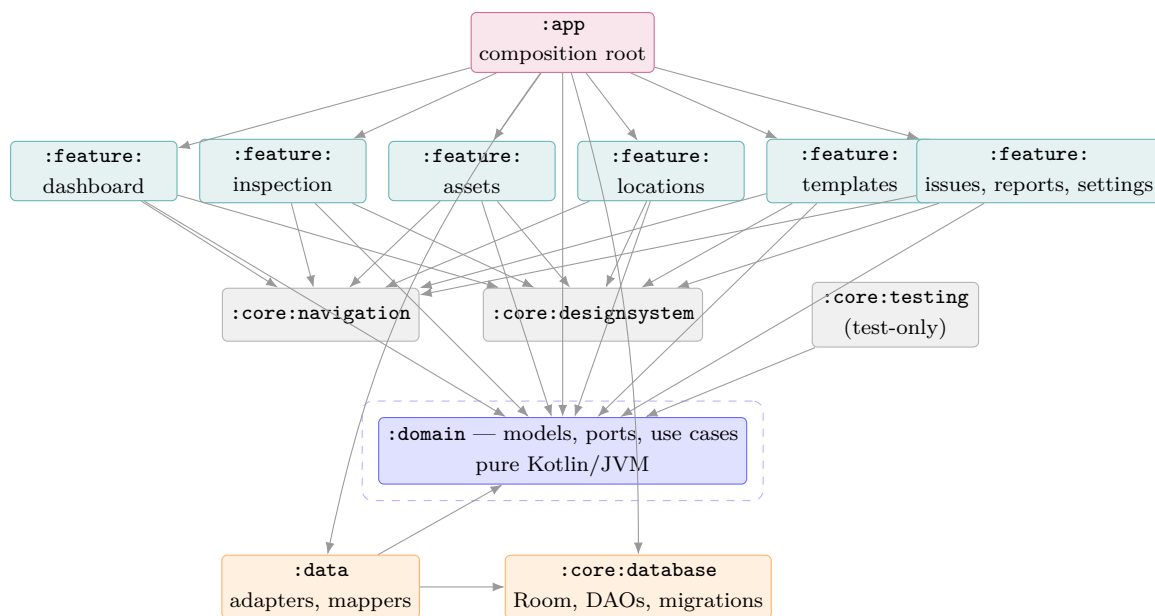


Figure 2: The FieldFlow module graph (15 Gradle modules). Every arrow is a compile-time dependency. `:domain` is the sink of the graph: many modules depend on it, it depends on nothing in the project.

The essential shape of Figure 2 is:

Feature → Domain ← Data

Feature modules and the data layer both point at the domain. Neither can see the other. `:app` sits above everything because it is the only module allowed to know concrete implementations.

3.2 Boundary rules

The module graph is governed by rules that are documented in `docs/architecture/MODULE_GRAPH.md` and enforced by the build files:

Allowed	Forbidden
<code>:app</code> may depend on everything.	Feature modules must not depend on <code>:data</code> or <code>:core:database</code> .
Features may depend on <code>:domain</code> , <code>:core:navigation</code> , <code>:core:designsystem</code> .	Features must not import repositories, Room entities/DAOs, file adapters, sync adapters, or exporters.
<code>:data</code> may depend on <code>:domain</code> and <code>:core:database</code> .	<code>:domain</code> must not import Android, Compose, Circuit, Room, app, feature, or data packages.
<code>:domain</code> may use Coroutines Flow (ports are reactive).	Production source sets must not depend on <code>:core:testing</code> ; only <code>:app</code> may assemble concrete adapters.

Table 2: Boundary rules for the FieldFlow module graph.

3.3 The domain layer

`:domain` contains 23 model files, 8 repository ports and 18 use-case files (1,789 lines of production Kotlin). It has no Android code and can be compiled and tested on a plain JVM.

3.3.1 Business rules, and why they live here

The rules FieldFlow owns are the reason the architecture pays for itself:

Rule	Location
A required checklist item must be answered before completion	ValidateInspectionUseCase.kt
A critical item answered Fail must carry evidence	ValidateInspectionUseCase.kt
Score is weighted; NotApplicable is excluded from the denominator	CalculateInspectionScoreUseCase.kt
Only valid lifecycle transitions may complete an inspection	CompleteInspectionUseCase.kt
Every critical failure creates a maintenance issue	CreateMaintenanceIssueUseCase.kt
The next due date follows the template's, then the asset's, recurrence policy	ScheduleNextInspectionUseCase.kt
A report may only be generated from a completed inspection	GenerateInspectionReportUseCase.kt

Table 3: Business rules implemented in :domain.

Two of these rules, in their actual implementation:

```
class ValidateInspectionUseCase {

    operator fun invoke(
        session: InspectionSession,
        template: InspectionTemplate,
    ): ValidationResult {
        val errors = mutableListOf<InspectionValidationError>()
        // ... (Data mapping for allItems and answerMap omitted) ...

        for ((itemId, item) in allItems) {
            val answer = answerMap[itemId]

            for ((itemId, item) in allItems) {
                val answer = answerMap[itemId]

                // RULE 2: Critical + Fail must have evidence
                if (item.critical && answer?.value == ChecklistAnswerValue.Fail) {
                    if (answer.evidenceIds.isEmpty()) {
                        errors.add(InspectionValidationError.CriticalFailureNeedsEvidence(
                            itemId = itemId,
                            message = "Critical item requires evidence on failure."
                        ))
                    }
                }
            }
        }

        // ... (Rule 1 omitted for brevity) ...
    }
    return ValidationResult(isValid = errors.isEmpty(), errors = errors)
}
```

Listing 3: domain/.../usecase/ValidateInspectionUseCase.kt — a pure function over two domain models. No repository, no coroutine, no Android. This is the easiest kind of code to test and the hardest kind to break accidentally.

```
for (section in template.sections) {
    for (item in section.items) {
        val answer = answerMap[item.id]

        // RULE 6: NotApplicable is excluded from the denominator
        if (answer?.value == ChecklistAnswerValue.NotApplicable) continue

        // RULE 5: Pass / Yes(true) earns weight, Fail / No(false) does not
        val scored = when (val value = answer?.value) {
            is ChecklistAnswerValue.Pass -> true
            is ChecklistAnswerValue.YesNo -> value.value
            else -> false
        }
        totalWeight += item.weight
        if (scored) earnedWeight += item.weight
    }
}
```

Listing 4: domain/.../usecase/CalculateInspectionScoreUseCase.kt — weighted scoring with NotApplicable excluded from the denominator. Expressing this in SQL inside a DAO would hide a business decision inside an infrastructure detail.

3.3.2 Ports

The domain declares eight ports. Each expresses a business need, not a storage technology:

Port	Business need it expresses
InspectionRepository	observe summaries and one session, create, save draft, complete
TemplateRepository	observe/load/save inspection templates
AssetRepository	observe assets and locations, load and save them
IssueRepository	observe/create/update maintenance issues
ReportRepository	observe and persist export history
ReportExporter	turn a report into an exported artefact
EvidenceStore	persist and delete evidence attached to answers
AppearancePreferenceRepository	observe and set the theme mode

Table 4: Domain ports. Note that none of the names mention Room, files, JSON, PDF or Android.

```
interface InspectionRepository {
    fun observeInspectionSummaries(): Flow<List<InspectionSummary>>
    fun observeInspection(inspectionId: InspectionId): Flow<InspectionSession?>
    suspend fun getInspection(inspectionId: InspectionId): InspectionSession?
    suspend fun createInspection(
        assetId: String, assetName: String, templateId: String, startedAtMillis: Long,
    ): InspectionId
    suspend fun saveDraft(session: InspectionSession)
    suspend fun complete(completed: CompletedInspection)
}
```

Listing 5: domain/.../repository/InspectionRepository.kt — the port. Flow is the only framework type it mentions.

3.3.3 Orchestration

CompleteInspectionUseCase is the clearest example of a use case as an *application* rule: it sequences other rules but contains no infrastructure.

```
class CompleteInspectionUseCase(
    private val inspectionRepository: InspectionRepository,
    private val templateRepository: TemplateRepository,
    private val assetRepository: AssetRepository,
    private val validateInspection: ValidateInspectionUseCase,
    private val calculateScore: CalculateInspectionScoreUseCase,
    private val createIssue: CreateMaintenanceIssueUseCase,
    private val scheduleNext: ScheduleNextInspectionUseCase,
) {
    suspend operator fun invoke(
        inspectionId: InspectionId,
        completedAtMillis: Long = System.currentTimeMillis(),
    ): CompleteInspectionResult {
        val session = inspectionRepository.getInspection(inspectionId)
        ?: return CompleteInspectionResult.Error("Inspection not found")
        val template = templateRepository.getTemplate(session.templateId)
        ?: return CompleteInspectionResult.Error("Template not found")
        val asset = assetRepository.getAsset(session.assetId)
        ?: return CompleteInspectionResult.Error("Asset not found")

        if (session.status != InspectionStatus.REVIEWING &&
            session.status != InspectionStatus.IN_PROGRESS) {
            return CompleteInspectionResult.ValidationFailed(listOf(
                InspectionValidationError.InvalidLifecycleTransition(
                    from = session.status, to = InspectionStatus.COMPLETED, /* ... */)
            ))
        }

        val validation = validateInspection(session, template)
        if (!validation.isValid) return CompleteInspectionResult.ValidationFailed(validation.errors)

        val score = calculateScore(session, template)
        val newIssues = createIssue(findCriticalFailures(session, template), completedAtMillis)
        val nextDue = scheduleNext(template, asset, completedAtMillis)

        inspectionRepository.complete(CompletedInspection(
            id = session.id, answers = session.answers, score = score, issues = newIssues,
            nextInspectionDueAtMillis = nextDue, completedAtMillis = completedAtMillis))

        return CompleteInspectionResult.Success(score, newIssues, nextDue, completedAtMillis)
    }
}
```

Listing 6: domain/.../usecase/CompleteInspectionUseCase.kt (abridged) — all seven collaborators are domain types; none is a Room type.

3.4 The data layer

:core:database owns the Room boundary: 11 entities, 5 DAOs, explicit migrations, and `exportSchema = true` so schema JSON is versioned in git.

```
@Database(
    entities = [
        LocationEntity::class, AssetEntity::class,
        InspectionTemplateEntity::class,
        // ... (8 other entities omitted for brevity) ...
        PendingSyncEntity::class,
    ],
    version = 3,
    exportSchema = true, // Schema versioned in Git
)
abstract class FieldFlowDatabase : RoomDatabase() {
    abstract fun catalogDao(): CatalogDao
    abstract fun inspectionDao(): InspectionDao
    // ... (Other DAOs omitted) ...
}
```

Listing 7: core/database/.../FieldFlowDatabase.kt (abridged).

:data adapts that boundary to the domain ports. The canonical path is:

Room entity → mapper → domain model → Flow → use case

A Room entity never escapes :data. The mapping is explicit, in five mapper files, and this is a deliberate cost we accept in exchange for keeping storage shape and business shape independent.

```
class RoomInspectionRepository(
    private val database: FieldFlowDatabase,
    private val inspectionIdFactory: () -> String = { "inspection-${UUID.randomUUID()}" },
) : InspectionRepository { // Implementing the Domain Port

    private val inspectionDao = database.inspectionDao()

    override fun observeInspectionSummaries(): Flow<List<InspectionSummary>> =
        inspectionDao.observeInspectionSummaries()
        // Mapping infrastructure models to pure domain models
        .map { records -> records.map { it.toDomain() } }
        .distinctUntilChanged()

    override fun observeInspection(inspectionId: InspectionId): Flow<InspectionSession?> =
        inspectionDao.observeInspectionSession(inspectionId.value)
        .flatMapLatest { record ->
            if (record == null) flowOf(null)
            else combine(
                // Composing multiple DAOs into a single domain aggregate
                inspectionDao.observeAnswers(inspectionId.value),
                inspectionDao.observeEvidence(inspectionId.value),
            ) { answers, evidence ->
                record.toDomain(answers, evidence)
            }
        }
        .distinctUntilChanged()
}
```

Listing 8: `data/.../RoomInspectionRepository.kt` (abridged) — the adapter composes DAO flows and maps records to domain models. The class implements a domain interface, which is what makes the dependency point inward.

`:data` also supplies `AndroidEvidenceStore` (file-backed evidence), `JsonReportExporter` and `PdfReportExporter` behind the single `ReportExporter` port, `FieldFlowSampleDataSeeder`, and a deterministic `FakeRemoteSyncAdapter`.

3.5 The presentation layer

Each feature module owns one slice of presentation and contains the same five parts: a contract (state + events), a presenter, a Compose UI, Circuit factories, and tests. Screen contracts themselves live in `:core:navigation`, so features never import each other.

```
@Immutable
sealed interface InspectionState : CircuitUiState {
    val eventSink: (InspectionEvent) -> Unit

    data class Loading(override val eventSink: (InspectionEvent) -> Unit) : InspectionState

    data class Editing(
        val title: String,
        val sections: List<InspectionSectionUi>,
        val currentSectionIndex: Int,
        val progress: InspectionProgress,
        val saveError: String? = null,
        override val eventSink: (InspectionEvent) -> Unit,
    ) : InspectionState

    data class Reviewing(/* ... */) : InspectionState
    data class ValidationFailed(
        val errors: List<ValidationError>, /* ... */) : InspectionState
    data class Completed(/* ... */) : InspectionState
}
```

Listing 9: `feature/inspection/.../InspectionContracts.kt` (abridged) — the state machine is explicit and immutable.

The presenter is where the architecture is most visible, because of what is *absent* from its constructor:

```
/** Coordinates the inspection workflow without owning business rules. */
internal class InspectionPresenter(
    private val screen: InspectionScreen,
    private val navigator: Navigator,
    private val observeInspection: ObserveInspectionUseCase,
    private val observeTemplate: ObserveTemplateUseCase,
    private val saveInspectionDraft: SaveInspectionDraftUseCase,
    private val validateInspection: ValidateInspectionUseCase,
    private val completeInspection: CompleteInspectionUseCase,
) : Presenter<InspectionState> {

    @Composable
    override fun present(): InspectionState {
```

```

    val sessionWithTemplate by remember(screen.inspectionId) {
        observeInspection(InspectionId(screen.inspectionId))
            .flatMapLatest { session ->
                if (session == null) flowOf(null to null)
                else observeTemplate(session.templateId).map { session to it }
            }
    }.collectAsState(initial = null to null)
    // ... maps domain session + template onto immutable UI state
}
}

```

Listing 10: `feature/inspection/.../InspectionPresenter.kt` (abridged). Five collaborators, all of them domain use cases. There is no repository, no DAO, no `Context`. The comment in the source — “Coordinates the inspection workflow without owning business rules” — is enforced by the module graph, not just asserted.

3.6 The composition root

`FieldFlowCompositionRoot.kt`, in `:app`, is the only place in the codebase that knows concrete implementations. It opens Room with migrations, constructs the adapters, injects them into domain use cases, registers Circuit factories, schedules sample-data seeding on an app-scoped IO coroutine so that `onCreate` is never blocked by database work, and starts the pending-sync drain on that same scope.

The graph is owned by `FieldFlowApplication`, not by `MainActivity`: it holds the Room handle and an application-scoped `CoroutineScope`, so it must outlive Activity recreation. Creating it in the `Application` means a configuration change no longer closes and reopens the database or cancels an in-flight draft save.

```

fun create(applicationContext: Context): FieldFlowCompositionRoot {
    val database = Room.databaseBuilder(
        applicationContext, FieldFlowDatabase::class.java, FieldFlowDatabase.DATABASE_NAME,
    ).addMigrations(*FieldFlowMigrations.ALL).build()

    // Application-scoped CoroutineScope to outlive Activity recreation
    val appScope = CoroutineScope(SupervisorJob() + Dispatchers.IO)

    return try {
        val syncDao = database.syncDao()
        // Substituting a fake adapter since the backend doesn't exist yet
        val remoteSyncAdapter = FakeRemoteSyncAdapter(
            syncDao = syncDao,
            scenario = AttemptBasedFakeSyncScenario(outcomesByAttempt = emptyMap()),
            clock = System.currentTimeMillis
        )

        createWithRepositories(
            database = database,
            appScope = appScope,

            // Injecting concrete implementations (Adapters) into the graph...
            inspectionRepository = RoomInspectionRepository(database),
            templateRepository = RoomTemplateRepository(database.catalogDao()),
            // ... (Other repositories omitted for brevity) ...

            seedSampleData = { FieldFlowSampleDataSeeder(database).seedIfEmpty() },
        )
    }
}

```

```

        pendingSyncDrain = PendingSyncDrain(
            retryableCommands = syncDao.observeRetryableCommands(),
            sync = remoteSyncAdapter::sync
        )
    }
} catch (throwable: Throwable) {
    // Prevent leaking the scope and the open database handle on failure
    appScope.cancel()
    database.close()
    throw throwable
}
}

// ... later in the graph instantiation ...

// Use cases are built relying strictly on abstract *ports* (interfaces),
// completely unaware of the Room database or Android framework.
val completeInspection = CompleteInspectionUseCase(
    inspectionRepository = inspectionRepository, // Type: InspectionRepository
    templateRepository   = templateRepository,  // Type: TemplateRepository
    assetRepository      = assetRepository,
    validateInspection   = validateInspection,
    calculateScore       = CalculateInspectionScoreUseCase(),
    createIssue          = CreateMaintenanceIssueUseCase(),
    scheduleNext         = ScheduleNextInspectionUseCase(),
)

```

Listing 11: FieldFlowCompositionRoot.kt (abridged) — the single point at which “which adapter?” is answered.

The private helper `createWithRepositories` takes *port* types as parameters. That signature is what makes the whole graph substitutable: the same function can be called with different adapters, and nothing else in the project changes.

3.7 Offline-first

Room is the local source of truth. Reads are Flow-based, so a write in one screen propagates to every observer automatically. Drafts are persisted, so an inspection survives process death. A `PendingSyncEntity` table records work that a future backend would replay: completing an inspection enqueues a `PENDING` command, and the inspection reports as `InspectionStatus.SYNC_PENDING` until that command settles.

That queue now has a consumer. `PendingSyncDrain` (in `:app`) collects `SyncDao.observeRetryableCommands()` on the app scope and hands each command to `FakeRemoteSyncAdapter`, with exponential backoff capped at 60 seconds and a bounded attempt count (`DEFAULT_MAX_SYNC_ATTEMPTS = 5`) after which a command is left `FAILED` rather than retried forever. Two details are worth noting because they are the kind of defect a diagram cannot show. Without the drain, nothing ever consumed the queue — every completed inspection stayed `PENDING`, so the Dashboard “Sync pending” tile only counted up. And because a sync is two writes (claim, then settle), the pair runs inside `NonCancellable`: cancelling between them would park the command in `SYNCING`, a state `observeRetryableCommands` never returns, stranding the inspection with nothing left to retry it.

The adapter is still a fake — there is no server — but the *path* is live, and it is exercised end to end against a real Room database in `PendingSyncDrainEndToEndTest.kt`. Offline-first is therefore an *architectural* property here: the domain never knows whether data came from cache or network, because it only ever sees domain models arriving through a port. Replacing the fake with a real client is a change in `:data` plus one argument in the composition root.

4 Demo and Case Study

4.1 The application

FieldFlow runs as a Compose application with a Circuit navigation shell. The implemented, Room-backed workflows are: Dashboard, Inspection, Assets, Locations, Templates, Issues, Reports, and Settings.

Screen	Implemented behaviour
Dashboard	overview metrics (total / in-progress / sync-pending / open issues), continue-inspection card, status filters, start-inspection sheet that picks an asset and template and creates the draft
Inspection	sections, pass/fail/NA/measured answers, notes, evidence capture, draft save (including on system back), review, validation, completion
Assets	list, detail, add/edit, location association, validation, start-inspection handoff
Locations	list, search, detail, add/edit, validation (deletion omitted to protect asset associations)
Templates	list, detail with sections/items, add template with an initial checklist item, metadata editing that preserves the checklist aggregate
Issues	list with filters, detail, domain-validated lifecycle transitions
Reports	completed-inspection candidates, report detail, JSON and PDF export, persisted export history, open/share
Settings	System / Light / Dark appearance, persisted and applied at the theme root

Table 5: Implemented FieldFlow workflows.

4.2 End-to-end trace: completing an inspection

This is the flow we demonstrate live, because it crosses every boundary in the architecture exactly once.

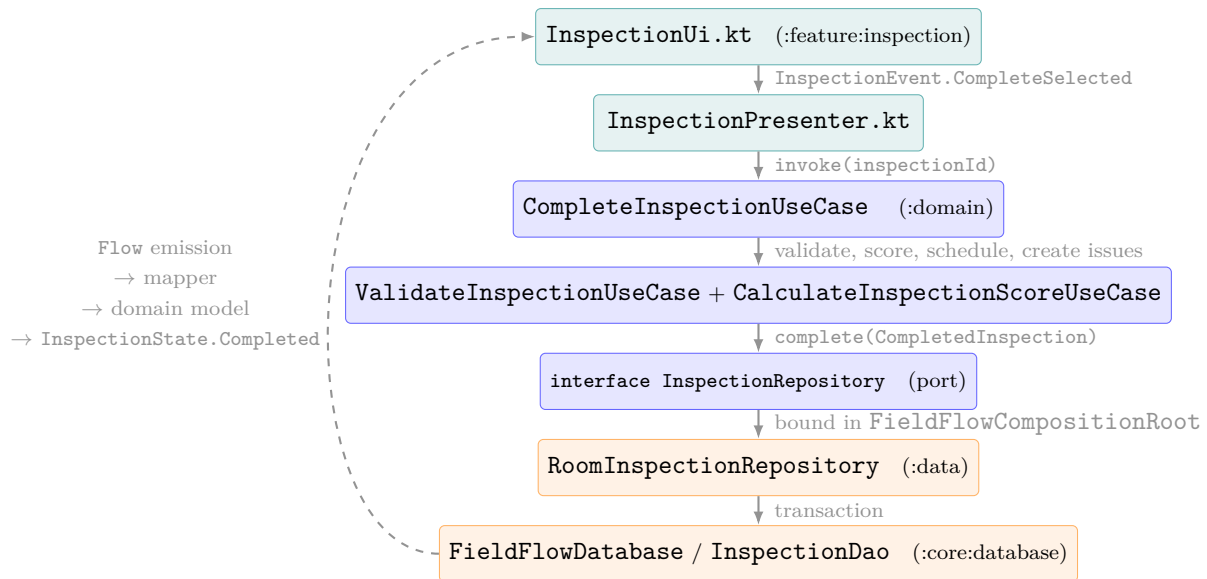


Figure 3: The inspection completion flow. Each downward arrow crosses exactly one architectural boundary; the dashed arrow is the reactive return path.

Two observations make this the core of the demonstration:

- The presenter never decides whether the inspection is valid. It calls a use case and renders the result. Moving the validation rule requires editing one domain file, and no presentation file.
- Nothing above `InspectionRepository` in Figure 3 knows that Room exists. Swapping persistence means editing `FieldFlowCompositionRoot.kt` and adding one class in `:data`.

4.3 Verification method

We verify the architecture at five levels rather than by inspection alone.

1. **Compile-time boundary checks.** The build itself is the primary test. If a feature module imported `:data`, compilation would fail. We verify the negative by reading the build files (no `:data` dependency in any `feature/*/build.gradle.kts`) and by the fact that the project compiles.
2. **Pure JVM domain tests.** `:domain` tests run with `:domain:test` — a plain `java-library` test task, with no emulator, no Robolectric, and no Android SDK. That this task exists at all is evidence of domain purity.
3. **Adapter and database tests.** DAO behaviour, migrations, draft recovery, pending-sync state, mappers, exporters and seeding are tested in `:core:database` and `:data`.
4. **Presenter tests.** Each feature’s presenter is tested against fakes from `:core:testing` using `circuit-test`, verifying state transitions without rendering Compose.
5. **Instrumented tests in CI.** Four navigation and draft-persistence tests run on a Gradle-managed device (Pixel 6, API 35, `aosp-atd`) declared in `app/build.gradle.kts`, so

`:app:ciDeviceDebugAndroidTest` is the same command locally and on CI. GitHub Actions runs it on every push and pull request to `main`.

The device is pinned to API 35 rather than 36, which would match `targetSdk`, for two measured reasons: the ATD image only reaches the stable SDK channel up to 35, and on the API 36 image the two `Espresso.pressBack()` tests fail headless with `RootViewWithoutFocusException` because the app window never takes focus. The cost, stated plainly, is that behaviour specific to targeting API 36 goes uncovered by CI.

Screen-level acceptance beyond those four tests is still performed by hand using `docs/demo/FINAL_MANUAL_ACCEPTANCE_CHECKLIST.md`.

The verification command used for the results below is:

```
.\gradlew.bat :domain:test :data:testDebugUnitTest :core:database:testDebugUnitTest ^
:app:testDebugUnitTest :feature:dashboard:testDebugUnitTest ^
:feature:assets:testDebugUnitTest :feature:locations:testDebugUnitTest ^
:feature:templates:testDebugUnitTest :feature:inspection:testDebugUnitTest ^
:feature:issues:testDebugUnitTest :feature:reports:testDebugUnitTest ^
:feature:settings:testDebugUnitTest --no-daemon
```

Listing 12: Module-scoped verification (PowerShell).

4.4 Results

Module	Files	Prod. LOC	Test LOC	Tests
:domain	54	1,789	952	34
:data	28	1,992	1,134	35
:core:database	16	1,125	674	11
:core:navigation	1	309	–	–
:core:designsystem	7	531	–	–
:core:testing	5	430	–	–
:app	14	957	600	16
:feature:dashboard	11	1,987	570	20
:feature:inspection	7	1,397	1,265	40
:feature:assets	5	1,211	233	6
:feature:locations	5	905	116	3
:feature:templates	5	1,446	286	8
:feature:issues	5	885	187	5
:feature:reports	5	1,248	291	5
:feature:settings	5	349	129	4
Total	173	16,561	6,437	187

Table 6: Measured size of the FieldFlow codebase. The **Files** column counts production plus unit-test Kotlin files; **Test LOC** and **Tests** cover unit tests only. The suite was executed with `--rerun-tasks` on 27 July 2026: 31 suites, 187 tests, 0 failures, 0 errors, 0 skipped, `BUILD SUCCESSFUL` in 3 min 53 s. Instrumented code is counted separately: `FieldFlowNavigationSmokeTest.kt`, 223 lines, 4 tests, which are not part of the 187.

The numbers that matter architecturally are not the totals but their *distribution*:

- **Test-to-production ratio in `:domain` is 0.53** (952 test lines to 1,789 production lines), second only to `:feature:inspection` at 0.91. That density is only achievable because domain tests need no Android runtime.
- **34 of the 187 tests run on a bare JVM** through `:domain:test`. Every other module's unit tests are Android unit tests, and the ones that touch Room or files (`:data`, `:core:database`, `:app`) need Robolectric. If business rules lived in composables, those 34 tests would need it too.
- **`:domain` has one production dependency.** Its 1,789 lines are coupled to `kotlinx-coroutines-core` and nothing else.
- **Zero feature modules depend on `:data`.** This was verified across all eight `feature/*/build.gradle.kts` files.

Representative domain test names, taken directly from `DomainBusinessRulesTest.kt`, show that the tests are written in the language of the business rather than the language of the framework:

```
fun 'cannotCompleteWithUnansweredRequiredItem'()
fun 'criticalFailureRequiresEvidence'()
fun 'criticalFailureCreatesIssue'()
fun 'notApplicableIsExcludedFromScore'()
fun 'invalidLifecycleTransitionIsRejected'()
fun 'reportRequiresCompletedInspection'()
fun 'nextInspectionDateUsesRecurrencePolicy'()
fun 'nextInspectionDateFallsBackToAssetPolicyWhenTemplateHasNoPolicy'()
fun 'critical issue IDs remain stable when completion is retried'()
```

Listing 13: `domain/src/test/.../DomainBusinessRulesTest.kt` — test names as executable specification.

4.5 Case study: replacing an adapter

The strongest practical claim of Clean Architecture is replaceability, so we tested it rather than asserting it. FieldFlow ships more than one adapter behind the same ports:

- the five `Room*Repository` adapters (`Inspection`, `Template`, `Asset`, `Issue`, `Report`) — the product runtime;
- `FakeInspectionRepository` (in `:data`) and the four fakes in `:core:testing` (`FakeAssetRepository`, `FakeTemplateRepository`, `FakeIssueRepository`, `RecordingInspectionRepository`) — deterministic in-memory adapters used by presenter tests and adapter-swap experiments;
- two exporters, `JsonReportExporter` and `PdfReportExporter`, behind the single `ReportExporter` port — the clearest case, because both are production adapters and the domain distinguishes them only by a `ReportFormat` argument on `export`.

Switching the inspection adapter means passing a different argument to `createWithRepositories`. The diff touches `:app` only. `:domain` is unchanged; all eight feature modules are unchanged; every use case is unchanged; and the presenter tests continue to pass because they

were never coupled to storage in the first place. This is dependency inversion producing a concrete, measurable result rather than a diagram.

We should be precise about the limit of this claim. An earlier revision of the project carried a parallel `DemoRepositories.kt`, a complete second adapter set for every port, and that file has since been deleted, because once the Room adapters worked it was dead weight that still had to compile. So what we demonstrate today is one port swapped at a time against a fake, plus one port (`ReportExporter`) with two live implementations. That is enough to show the seam is real; it is not the same as running the whole application on a second persistence stack, and we do not claim it is.

5 Evaluation

5.1 Advantages observed

Benefit	Evidence in this codebase
Separation of concerns	Rules live in <code>ValidateInspectionUseCase.kt</code> ; <code>InspectionUi.kt</code> only renders state.
Testability	34 domain tests run on a bare JVM via <code>:domain:test</code> , with no Android runtime; 187 unit tests pass in under 4 minutes.
Replaceability	Room adapters and in-memory fakes satisfy the same ports, and JSON/PDF exporters share one port; switching touches <code>:app</code> only.
Compile-time safety	No <code>feature/*/build.gradle.kts</code> declares <code>:data</code> ; violations fail the build.
Framework isolation	<code>:domain</code> has one production dependency and no Android, Compose, Circuit, or Room import.
Parallel development	Four members owned <code>:app:feature:inspection</code> , <code>:domain</code> , <code>:data:core:database</code> , and <code>dashboard/reports/design</code> system, meeting only at domain contracts.
Scalability	Adding <code>:feature:settings</code> required a new module, a screen contract, and two composition-root lines — no change to existing features.

Table 7: Benefits, each tied to something checkable in the source tree.

The parallel-development benefit deserves emphasis because it is the one most often claimed and least often examined. FieldFlow’s four members agreed the domain contracts first and then worked on role-scoped branches — the git history shows `feat/domain-business-rules`, `codex/data-room-foundation` and `feat/inspection-workflow` being merged in dependency order (`domain` → `data` → `features` → `integration`) rather than in completion order. The only shared file set was `:domain`, so contract changes were the one thing that required cross-role review. We should be candid that this was not frictionless: the history also contains several explicit conflict-resolution commits, mostly around integration points.

5.2 Limitations and costs

We do not claim the architecture is free. The measured and observed costs are:

- **Structural overhead.** 15 Gradle modules, 15 build scripts, and a version catalog to keep consistent. Configuration time and IDE indexing both grow with module count.
- **Mapping cost.** Five mapper files exist solely to translate Room records into domain models. This is duplicated shape — an entity and a model that often look similar — and it must be maintained in step.
- **Indirection.** Reading the completion flow requires opening a presenter, a use case, a port, an adapter, and a DAO. For a newcomer, "where does this actually happen?" is a real and recurring question.
- **Composition-root growth.** `FieldFlowCompositionRoot.kt` is 349 lines of manual wiring with no dependency-injection framework, and `createWithRepositories` now takes 13 parameters. It is explicit and testable, but it grows linearly with every feature and is the single most conflict-prone file in the repository.
- **Learning curve.** Circuit plus Clean Architecture plus multi-module Gradle is three unfamiliar things at once. The cost was paid during the first days of the project, not amortised over it.
- **Enforcement has gaps.** Gradle prevents feature → data imports, but nothing mechanically prevents a business rule from being written inside a presenter. That boundary is still maintained by review and by convention.
- **Verification is still unit-heavy.** 187 unit tests against 4 instrumented tests. CI now runs the instrumented suite on a managed device on every push, which closed the worst of the gap, but those four tests cover navigation and draft persistence only. Most screen-level behaviour across the eight features is still confirmed by hand, and the API 35 device means API 36-specific behaviour is uncovered.
- **The offline-first story is only half-proven.** `PendingSyncDrain` is wired and tested end to end, but against `FakeRemoteSyncAdapter`. Conflict resolution, partial failure from a real server, and authentication are the parts of synchronisation that are genuinely hard, and none of them has been attempted here.

5.3 Trade-offs

Trade-off	What we gained	What we paid
Modules vs. packages	compile-time enforcement, clear ownership	build configuration, slower cold builds
Ports vs. direct calls	replaceable storage, JVM-testable rules	one interface + one adapter per capability
Mappers vs. shared models	storage schema and business model evolve independently	duplicated shape, mapping code to maintain
Use cases vs. fat presenters	rules testable and reusable across screens	more small classes, more indirection
Manual DI vs. Hilt/Dagger	no annotation processing, explicit and debuggable graph	a large hand-written root that grows with the app

Table 8: The principal trade-offs, stated in both directions.

The manual-DI choice is worth expanding. Hilt would shrink `FieldFlowCompositionRoot.kt` substantially and remove the conflict hot-spot. We chose manual wiring because it makes the dependency graph readable in one file — valuable for a seminar whose subject *is* the dependency graph — and because it adds no KSP round to the build. For a production app of this size, Hilt would likely be the better choice.

5.4 Alternatives considered

Alternative	Why it is attractive	Why we did not choose it
Single-module MVVM, ViewModel calls repository directly	fewest files; fastest to a running screen; idiomatic Android	validation, scoring, evidence and lifecycle rules would sit in ViewModels, needing Robolectric to test and being copied across screens
Layered packages in one module (<code>ui/</code> , <code>domain/</code> , <code>data/</code>)	expresses the same layering with no Gradle overhead	the layering is unenforced; a single stray import silently violates it, and nothing fails
MVI with a reducer, no use-case layer	excellent state predictability; one obvious place per screen	rules end up inside reducers, so they are per-screen rather than shared, and cross-screen rules such as report eligibility have no home
Clean Architecture with Hilt instead of manual DI	removes the hand-written composition root	adds an annotation-processing round and hides the dependency graph the seminar needs to show; a reasonable choice in production

Table 9: Alternatives, and the specific reason each was rejected *for this project*.

5.5 Conditions for use

Clean Architecture is not universally correct, and the honest conclusion of this case study is a conditional one.

Use it when the application owns non-trivial business rules that outlive any single screen; when storage or delivery mechanisms are expected to change; when several people must work in parallel without stepping on each other; when rules must be tested cheaply and repeatedly; or when the product is expected to live for years.

Do not use it when the application is essentially a UI over a table with no rules of its own; when it is a prototype whose main risk is not shipping at all; when the team is one person on a short deadline; or when nobody on the team has used the pattern before and the deadline leaves no room to learn it. In those cases a single module with ViewModels and a repository is the better engineering decision, and adopting Clean Architecture would be over-engineering.

FieldFlow sits clearly on the first side of that line: it has seven documented business rules, offline-first persistence, replaceable export and evidence adapters, four parallel owners, and an explicit list of future integrations. That is why the cost is justified here — and why we would not claim it is justified everywhere.

6 Conclusion

6.1 Key findings

1. **The Dependency Rule becomes real only when a tool enforces it.** The single highest-value decision in this project was making `:domain` a `java-library` rather than an Android library. That one line in a build file makes an entire category of architectural violation impossible to compile.
2. **Testability is a consequence of the boundary, not an extra effort.** We did not work to make the domain testable; it is testable because it has one dependency. 34 business-rule tests run in seconds on a plain JVM, inside a 187-test suite that completes in under 4 minutes.
3. **Replaceability is measurable.** Room adapters and in-memory fakes satisfy the same ports, two exporters share one port, and switching between any of them changes exactly one module `:app`.
4. **Circuit and Clean Architecture are orthogonal.** Circuit organises the presentation circle; Clean Architecture decides which circle owns policy. Conflating them, treating presenters as a place for rules, would have dissolved the boundary while leaving the diagram intact.
5. **The cost is real and concentrated.** It appears as 15 build scripts, five mapper files, and a 349-line composition root. It is justified by this domain; it would not be justified by a simple CRUD app.
6. **A port with no adapter running is not a feature.** The `pending_sync` table, its DAO and `FakeRemoteSyncAdapter` all existed and were unit-tested while nothing consumed the queue,

so every completed inspection sat on "Sync pending" forever. Clean Architecture makes the seam easy to fill correctly; it does nothing to tell you the seam is empty. Only running the app did.

6.2 Lessons learned

- **Agree the contracts before writing implementations.** Our team settled the IDs, models and repository ports on day one. Every later change went through the sequence domain $\rightarrow$ data $\rightarrow$ presentation. Contract-first work, more than the module graph itself, is what made four people working in parallel viable.
- **Merge in dependency order, not completion order.** Domain first, then data, then features, then integration. Merging by who finished first produced conflicts; merging by dependency did not.
- **Boundaries a tool cannot check will erode.** The feature $\rightarrow$ data boundary held perfectly because Gradle enforces it. The "no business rules in presenters" boundary required continuous review, because nothing enforces it.
- **Write the trade-offs down.** The most useful artefact for the seminar was not the diagram but the honest list of costs. An architecture presented without its price is not an argument.
- **Distinguish implemented from planned — and re-check the list.** We maintained a list of what is wired at runtime versus what exists only as infrastructure. `FakeRemoteSyncAdapter` sat on the second list for most of the project and has since moved to the first, which is exactly why the list has to be re-derived from the code rather than remembered. Claiming more than the code does, or less, would have undermined everything else in this report.

7 AI Usage Statement

Generative AI tools were used as *development assistants* during the implementation and documentation of FieldFlow. In particular, the team used **ChatGPT** and **OpenAI Codex** to support selected engineering tasks. AI was not treated as an authoritative source, and it did not replace the team's design decisions, implementation responsibility, or technical verification.

Scope of AI assistance. AI tools supported the project in the following areas:

- **Architecture discussion:** comparing design alternatives and reviewing the proposed Circuit-Based Feature-Modular Clean Architecture.
- **Implementation support:** drafting and refactoring selected Kotlin, Compose, Circuit, Room, Gradle, and test code.
- **Debugging and integration:** analysing compiler messages, test failures, Room migrations, module dependencies, and post-merge integration risks.

- **Testing support:** proposing unit-test cases, boundary checks, migration tests, presenter tests, and manual Android acceptance scenarios.
- **Documentation support:** improving the structure, wording, and readability of the report, architecture documents, demonstration script, and testing checklists.

Human review and verification. All AI-generated suggestions were treated as *proposals* rather than final answers. Before being accepted, each relevant suggestion was reviewed and adapted by the team to match the requirements, conventions, and architecture of FieldFlow.

The resulting implementation was verified through:

- Gradle compilation and module-boundary checks;
- pure JVM tests for domain rules;
- Room database, repository, mapper, and migration tests;
- Circuit presenter and application-level tests;
- instrumented navigation and persistence tests; and
- manual Android Studio acceptance testing.

Technical statements in this report, including module dependencies, implemented features, file locations, test counts, and build results, were checked against the actual source tree and build output. They were not accepted solely because they were suggested by an AI system.

Team responsibility. The team remained fully responsible for:

- defining the product requirements and project scope;
- selecting and applying the architecture;
- designing domain models, business rules, ports, and module boundaries;
- resolving merge conflicts and integration issues;
- reviewing, testing, and approving the final implementation;
- evaluating the limitations and trade-offs of the architecture; and
- preparing and approving the final report and demonstration.

References and architectural explanations were verified against the cited books, research articles, and official documentation. Any remaining errors, omissions, or incorrect conclusions are the responsibility of the authors.

References

- [1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA: Prentice Hall, 2017.
- [2] R. C. Martin, "The Clean Architecture," *The Clean Code Blog*, 2012. [Online]. Available: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- [3] A. Cockburn, "Hexagonal Architecture (Ports and Adapters)," 2005. [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture/>
- [4] Android Developers, "Guide to app architecture." [Online]. Available: <https://developer.android.com/topic/architecture>
- [5] Android Developers, "Guide to Android app modularization." [Online]. Available: <https://developer.android.com/topic/modularization>
- [6] Android Developers, "Save data in a local database using Room." [Online]. Available: <https://developer.android.com/training/data-storage/room>
- [7] Android Developers, "Migrate your Room database." [Online]. Available: <https://developer.android.com/training/data-storage/room/migrating-db-versions>
- [8] Android Developers, "Jetpack Compose." [Online]. Available: <https://developer.android.com/compose>
- [9] Slack, "Circuit — A Compose-driven architecture for Kotlin and Android applications." [Online]. Available: <https://slackhq.github.io/circuit/>
- [10] JetBrains, "Asynchronous Flow — Kotlin Coroutines documentation." [Online]. Available: <https://kotlinlang.org/docs/flow.html>
- [11] Gradle, "Structuring Projects with Gradle — Multi-Project Builds." [Online]. Available: https://docs.gradle.org/current/userguide/multi_project_builds.html
- [12] Google, "Now in Android — a fully functional Android app built entirely with Kotlin and Jetpack Compose," GitHub repository. [Online]. Available: <https://github.com/android/nowinandroid>

A Building and Running FieldFlow

Full setup instructions are in `SourceCode/README.md`. In summary:

Prerequisites. JDK 21; Android SDK platform `android-36.1`; Gradle wrapper 9.4.1 (included). Toolchain versions: AGP 9.2.1, Kotlin 2.2.10, KSP 2.3.10, Compose BOM 2026.02.01, Room 2.8.4, Circuit 0.33.1. Minimum supported device: API 24.

```
# list the module graph
.\gradlew.bat projects --no-daemon
```

```
# run every unit test (187 tests, all passing)
.\gradlew.bat :domain:test :data:testDebugUnitTest :core:database:testDebugUnitTest ^
:app:testDebugUnitTest :feature:dashboard:testDebugUnitTest ^
:feature:assets:testDebugUnitTest :feature:locations:testDebugUnitTest ^
:feature:templates:testDebugUnitTest :feature:inspection:testDebugUnitTest ^
:feature:issues:testDebugUnitTest :feature:reports:testDebugUnitTest ^
:feature:settings:testDebugUnitTest --no-daemon

# lint, test and assemble the debug APK (what CI's first job runs)
.\gradlew.bat lintDebug test assembleDebug :app:assembleDebugAndroidTest --no-daemon

# instrumented suite on the Gradle-managed device -- same command CI uses.
# Provisions a Pixel 6 / API 35 / aosp-atd emulator on first run.
.\gradlew.bat :app:ciDeviceDebugAndroidTest --no-daemon

# instrumented suite on an already-attached device or emulator
.\gradlew.bat connectedDebugAndroidTest --no-daemon
```

Listing 14: Build and verification commands (PowerShell).

No account, API key, or external service is required: FieldFlow is offline-first and seeds its own sample data on first launch.

B Key Files for the Demonstration

File	What it demonstrates
domain/build.gradle.kts	domain purity: a <code>java-library</code> , one dependency
feature/inspection/build.gradle.kts	the absent <code>:data</code> dependency
domain/.../ValidateInspectionUseCase.kt	business rules as pure functions
domain/.../InspectionRepository.kt	a port owned by the inner circle
data/.../RoomInspectionRepository.kt	an adapter implementing that port
app/.../FieldFlowCompositionRoot.kt	the single place that knows concrete types
app/.../PendingSyncDrain.kt	an outer-circle mechanism the domain never sees
feature/.../InspectionPresenter.kt	a presenter that owns no rules
domain/.../DomainBusinessRulesTest.kt	rules verified without Android

Table 10: Files to open during the live demonstration, in order.